API一覧対応表

メタデータ

- **作成日**: 2026-01-20
- **バージョン**: 2.18.0
- **最終更新**: 2026-03-18
- **担当**: Development Team

概要

このドキュメントは、mmdpfプロジェクトの全APIエンドポイントを一覧化し、実装ファイルとAPI設計書を対応付けます。

使用方法

1. **API名で検索**: 実装したいAPIをこの一覧から探す
2. **ルートファイルを特定**: 実装ファイルパスで該当ファイルを開く
3. **API設計書を参照**: 詳細仕様を確認
4. **実装状態を確認**: 実装済み/未実装を確認

📋 API一覧

1. 認証API（Authentication）

API設計書: api_auth.md
ルートファイル: `backend/src/routes/authRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー

1.1	POST	/api/v1/auth/login	初回ログイン（PINコード送信）	✅ 実装済み	AuthController.login
1.2	POST	/api/v1/auth/verify-pin	PINコード検証・本セッション生成	✅ 実装済み	AuthController.verifyPin
1.3	POST	/api/v1/auth/resend-pin	PINコード再送信	✅ 実装済み	AuthController.resendPin
1.4	POST	/api/v1/auth/reset-password-request	パスワードリセット要求	✅ 実装済み	AuthController.resetPasswordRequest
1.5	POST	/api/v1/auth/reset-password	パスワードリセット実行	✅ 実装済み	AuthController.resetPassword
1.6	POST	/api/v1/auth/logout	ログアウト	✅ 実装済み	AuthController.logout

ミドルウェアチェーン:

- 1.1, 1.4: 認証不要
- 1.2, 1.3: 仮セッショントークン必須
- 1.5: リセットトークン必須
- 1.6: JWT認証 + テナント分離

備考:

- JWT + Session Hybrid認証を使用
- PINコードは6桁数字、有効期限5分
- セッション有効期限はロール別（12〜48時間）

---

2. プロジェクト管理API（Project Management）

API設計書: api_project_management.md
ルートファイル: `backend/src/routes/projectRoutes.ts`
備考: 実装済み18/20エンドポイント（90%）

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
2.1	GET	/api/v1/projects	プロジェクト一覧取得	✅ 実装済み	ProjectController.list
2.2	POST	/api/v1/projects	プロジェクト作成	✅ 実装済み	ProjectController.create

2.3	GET	/api/v1/projects/:projectId	プロジェクト詳細取得	✅ 実装済み	ProjectController.getById
2.4	POST	/api/v1/projects/:projectName/checkout	チェックアウト	✅ 実装済み	ProjectController.checkout
2.5	POST	/api/v1/projects/:projectName/checkin	チェックイン	✅ 実装済み	ProjectController.checkin
2.6	GET	/api/v1/projects/:projectId/checkout-status	チェックアウト状態取得	✅ 実装済み	ProjectController.getCheckoutStatus
2.7	GET	/api/v1/projects/:projectName/files	ファイル一覧取得	✅ 実装済み	ProjectController.getProjectFiles
2.8	GET	/api/v1/projects/:projectName/tasks	タスク一覧取得	✅ 実装済み	ProjectController.getProjectTasks
2.9	POST	/api/v1/projects/:projectName/instructions	指示送信（タスク作成）	⏳ 未実装	ProjectController.createInstruction
2.10	GET	/api/v1/projects/:projectName/instructions	指示一覧取得	⏳ 未実装	ProjectController.listInstructions
2.11	POST	/api/v1/projects/search	プロジェクト詳細検索	⏳ 未実装	ProjectController.search
2.12	POST	/api/v1/projects/:projectName/publish	MaiML公開ジョブ投入	✅ 実装済み	ProjectController.publish
2.12b	POST	/api/v1/projects/:projectName/unpublish	MaiML非公開ジョブ投入	✅ 実装済み	ProjectController.unpublish
2.12c	GET	/api/v1/projects/:projectName/publish-status	公開ジョブステータス取得	✅ 実装済み	ProjectController.getPublishStatus
2.13	GET	/api/v1/projects/:projectName/worker-instructions	指示一覧取得（実験作業者用）	✅ 実装済み	ProjectController.getWorkerInstructions
2.14	PUT	/api/v1/projects/:projectName/instructions/:instructionId/confirm	指示確認完了	✅ 実装済み	ProjectController.confirmInstruction
2.15	PATCH	/api/v1/projects/:projectName/approval-request	承認依頼	✅ 実装済み	ProjectController.requestApproval
2.16	GET	/api/v1/projects/worker-accessible	実験作業者用プロジェクト一覧取得	✅ 実装済み	WorkerProjectController.listWorkerAccessible
2.17	GET	/api/v1/projects/:projectName/maiml	MAIMLファイル取得（プロジェクト）	✅ 実装済み	ProjectController.getProjectMaiml

ミドルウェアチェーン:

- すべて: `auth` → `tenantIsolation` → `permissionCheck`

- 権限: Manager以上（Worker は閲覧のみ可能）

備考:

- すべてのクエリに organization_id フィルタ必須
- S3操作には organization_prefix を強制適用
- チェックアウト/チェックインで排他編集制御
- トランザクション管理が必要な操作: 2.2, 2.9
- API 2.3, 2.6: 実装追加（設計書には未記載、後日設計書に追記予定）
- パスパラメータ: 一部API で `:projectId`（UUID）、一部で `:projectName`（文字列）を使用
- **API 2.17: MAIMLファイル取得（プロジェクト）**
  - 指示確認タブのフロー図表示で使用
  - 実験作業者も閲覧可能（experiment_worker権限）
  - S3から直接取得（file_metadataテーブル不使用）
  - 最新版のみ取得（アーカイブ非対応）
  - 詳細仕様: api_project_management.md API #15を参照

---

3. 共通API（Common / File Operations）

API設計書: api_common.md
ルートファイル: `backend/src/routes/fileRoutes.ts`（推定）

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
3.1	POST	/api/v1/files/upload	ファイルアップロード（汎用）	⏳ 未実装	FileController.upload
3.2	POST	/api/v1/files/download	ファイル/フォルダダウンロード	⏳ 未実装	FileController.download

ミドルウェアチェーン:

- すべて: `auth` → `tenantIsolation` → `permissionCheck`
- 権限: Manager以上

備考:

- contextパラメータで保存先を判定（project/format）
- フォルダダウンロード時はZIP生成（最大1GB、1000ファイル）
- 署名付きURL有効期限: 1時間（最大24時間）
- マルチテナント分離必須（organization_prefix強制付与）

---

4. フォーマット管理API（Format Management）

API設計書: api_format_management.md
ルートファイル: `backend/src/routes/formatRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
4.1	GET	/api/v1/formats	フォーマット一覧取得	✅ 実装済み	FormatController.getFormats
4.2	GET	/api/v1/formats/:folderName/files	フォーマットファイル一覧取得	✅ 実装済み	FormatController.getFormatFiles
4.3	POST	/api/v1/formats/:folderName/files	新規ファイル作成	✅ 実装済み	FormatController.createFormatFile
4.4	POST	/api/v1/formats/folders	新規フォーマットフォルダ作成（部門直下）	✅ 実装済み	FormatController.createFormatFolder
4.5	POST	/api/v1/formats/subfolders	フォーマット内サブフォルダ作成	✅ 実装済み	FormatController.createFormatSubfolder
4.6	PUT	/api/v1/formats/:formatId/files/:fileId	ファイル編集（保存）	⏳ 未実装	FormatController.updateFile

ミドルウェアチェーン:

- 4.1-4.2: `authMiddleware` → `checkPermission('040000', 0)`（読み取り専用）
- 4.3-4.5: `authMiddleware` → `checkPermission('040000', 1)`（書込可）
- 権限: 実験管理者メニュー（040000）

備考:

- ワークフローMAIML（.maiml）とテンプレートXML（.xml）を管理
- S3パス: `{organizationId}/formats/{departmentCd}/`

- API 4.1: フォーマット一覧取得
  - セッションから部門CD（departmentCd）を取得
  - 部門配下のフォルダ一覧を取得（material, condition, result優先ソート）
  - file_typeクエリパラメータでフィルタリング可能（all/xml/maiml）

- API 4.2: フォーマットファイル一覧取得（2026-02-19実装）
  - サブフォルダ対応（pathクエリパラメータでサブパス指定）
  - ファイルとフォルダを区別して返却
  - フォルダには配下のファイル数（fileCount）を含む
  - 「↑ 戻る」ボタン用の特殊フォルダをフロントエンドで追加

- API 4.3: 新規ファイル作成（2026-02-24実装）
  - XMLファイル: templateType自動判定（material/condition/result）
  - MAIMLファイル: 空のMAIMLテンプレート生成
  - S3に空ファイルをプレースホルダーとして作成

- API 4.4: 新規フォーマットフォルダ作成（2026-02-24実装）
  - 部門直下にフォーマットフォルダを作成
  - S3にプレースホルダーファイル（.placeholder）を作成

- API 4.5: フォーマット内サブフォルダ作成（2026-02-24実装）
  - 既存フォーマットフォルダ内にサブフォルダを作成
  - parentPath指定でネストしたサブフォルダも作成可能
  - S3にプレースホルダーファイル（.placeholder）を作成

- XMLバリデーション必須（XXE攻撃対策）
- マルチテナント分離必須（organization_id強制付与）

---

5. Maiml編集API（MAIML Editor）

API設計書: api_maiml_editor.md
ルートファイル: `backend/src/routes/maimlRoutes.ts`
備考: 実装済み20/20エンドポイント（100%）、Draft機能追加

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
5.1	POST	/api/v1/maiml/parse	MAIML解析（基本）	✅ 実装済み	maimlController.parse
5.2	POST	/api/v1/maiml/parse-petri-net-structure	ペトリネット構造解析（複数method対応）	✅ 実装済み	maimlController.parsePetriNetStructure
5.3	POST	/api/v1/maiml/save	MAIMLファイル保存	✅ 実装済み	maimlController.save
5.4	POST	/api/v1/maiml/load	MAIMLファイル読込	✅ 実装済み	maimlController.load
5.5	GET	/api/v1/maiml/list	MAIMLファイル一覧取得	✅ 実装済み	maimlController.list
5.6	POST	/api/v1/maiml/validate	MAIMLバリデーション	✅ 実装済み	maimlController.validate
5.7	POST	/api/v1/maiml/auto-correct	MAIML自動補正（リアルタイム）	✅ 実装済み	maimlController.autoCorrect
5.8	POST	/api/v1/maiml/correct	MAIML自動補正（基本）	✅ 実装済み	maimlController.correct
5.9	POST	/api/v1/maiml/session/start	編集セッション開始	✅ 実装済み	maimlController.startSession
5.10	POST	/api/v1/maiml/session/end	編集セッション終了	✅ 実装済み	maimlController.endSession
5.11	GET	/api/v1/maiml/files	ファイル一覧取得	✅ 実装済み	maimlController.getFiles
5.12	GET	/api/v1/maiml/file-content	ファイル内容取得	✅ 実装済み	maimlController.getFileContent
5.13	GET	/api/v1/maiml/health	ヘルスチェック	✅ 実装済み	maimlController.health
5.14	POST	/api/v1/maiml/draft	Draft保存	✅ 実装済み	maimlController.saveDraft
5.15	GET	/api/v1/maiml/draft/:draftKey	Draft読み込み	✅ 実装済み	maimlController.loadDraft
5.16	GET	/api/v1/maiml/drafts	Draft一覧取得	✅ 実装済み	maimlController.listDrafts
5.17	DELETE	/api/v1/maiml/draft/:draftKey	Draft削除	✅ 実装済み	maimlController.deleteDraft

5.18	GET	/api/v1/maiml/formats	名前空間テンプレートフォルダ＆ファイル一覧取得	⏳ 未実装	maimlController.getFormats

ミドルウェアチェーン:

- 5.1-5.9: `auth` → バリデーション
- 5.3-5.5, 5.8-5.9: organization_id必須（tenantIsolation相当）
- 5.10: 認証不要
- 権限: Manager以上

備考:

- ✅ Phase 4（2026-01-27）: サービス層実装完了
  - parseMaiML, parsePetriNetStructure, saveMaiML, loadMaiML
  - listMaiMLFiles, validateMaiML, correctMaiML（スタブ）
  - startEditSession, endEditSession
- ✅ Phase 6（2026-01-27）: API接続完全統合
  - 新API `/parse-petri-net-structure` 追加（複数method対応）
  - フロントエンドPetriNetEditorを新APIに接続
- 複数メソッド対応（1ファイルに複数の実験手順）
- Petri Net構造解析（method別のplace/transition/arc抽出）
- ステレオタイプ自動抽出（materialTemplate, conditionTemplate, resultTemplate）
- InstructionID自動紐付け
- 重複ID検証統合
- XMLバリデーション・自動補正機能
- バックアップ機能（archiveフォルダ）
- storageService統合（S3/Swift対応）
- 旧エンドポイント（`/api/worker/*`）は2026年7月21日廃止予定
- **API 5.18: 名前空間テンプレートフォルダ＆ファイル一覧取得**
  - `editorv2` の名前空間テンプレート取り込み機能（TemplateEditorDialog）で使用
  - 全部門横断で `formats/` 配下の `material/` `condition/` `result/` フォルダの `.xml` ファイルのみを返す
  - フォーマット管理API（4.1/4.2）はセッションの部門CD単位のため流用不可

- `storageService.listFiles(organizationId, 'formats/')` で全部門を横断取得
- レスポンス構造: 部門CD → フォルダ（material/condition/result） → ファイルの3階層
- 詳細仕様: api_maiml_editor.md API #18を参照

---

6. 進捗確認API（Progress Monitoring）

API設計書: api_progress_monitoring.md
ルートファイル: `backend/src/routes/progressRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
6.1	GET	/api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/details	タスク詳細取得	✅ 実装済み	ProjectController.getTaskDetail
6.3	GET	/api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/transitions	進捗データ取得	✅ 実装済み	ProgressController.getProgressData
6.4	GET	/api/v1/progress/projects/:projectName/worker-tasks	タスク一覧取得（実験作業者用）	✅ 実装済み	ProgressController.getWorkerTasks
6.5	PATCH	/api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/status	タスクステータス更新	✅ 実装済み	ProgressController.updateTaskStatus
6.6	PATCH	/api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status	トランジションステータス更新	✅ 実装済み	ProgressController.updateTransitionStatus
6.7	GET	/api/v1/progress/projects/:projectName/work-details	作業詳細一括取得	✅ 実装済み	ProgressController.getWorkDetails

ミドルウェアチェーン:

- すべて: `auth` → `tenantIsolation` → `permissionCheck`
- 権限: 6.1, 6.7（experiment_admin 読取専用）、6.3-6.4（experiment_worker 読取専用）、6.5-6.6（experiment_worker 書込可）

備考:

- メソッド別進捗集約
- タスク詳細情報（担当者、ステータス、実験記録）
- リアルタイム進捗反映

- 進捗情報キャッシュ（5分間）
- API 6.4: 実験作業者が自分に割り当てられたタスクの一覧を取得（プロジェクト名で特定）
- API 6.5: プロジェクト名、メソッドID、タスク連番でタスクを特定してステータス更新
- API 6.6: トランジションステータス更新（実験データ記録APIから移動、taskNoはリクエストボディで指定）

---

7. 問題管理API（Issue Management）

API設計書: api_issue_management.md
ルートファイル: `backend/src/routes/issueRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
7.1	POST	/api/v1/issues	問題報告	✅ 実装済み	IssueController.create
7.2	POST	/api/v1/issues/comments	コメント投稿	✅ 実装済み	IssueController.addComment
7.3	GET	/api/v1/issues	問題一覧取得	✅ 実装済み	IssueController.list
7.4	GET	/api/v1/issues/:issueId	問題詳細取得	✅ 実装済み	IssueController.getById
7.5	PATCH	/api/v1/issues/status	問題ステータス変更	✅ 実装済み	IssueController.updateStatus

ミドルウェアチェーン:

- すべて: `auth` → `tenantIsolation` → `permissionCheck`
- 権限: 7.1（全ユーザー）、7.2（experiment_worker のみ書込可）、7.3-7.4（全ユーザー）、7.5（experiment_worker のみ書込可）

備考:

- ✅ 実装完了（テスト103件すべてPASS）
- 問題の種類: material_shortage, equipment_failure, procedure_unclear, safety_concern, quality_issue, time_constraint, other
- 対象種別: project, task, transition
- 問題一覧取得（API 7.3）の仕様:

    - issue_target（問題対象）に基づく条件分岐フィルタリング
        - `project` : project_idのみで一致判定
        - `task` : project_id + method_id + task_noで一致判定
        - `transition` : project_id + method_id + transition_idで一致判定
    - 各問題に紐づくコメント（issue_comment）を親子関係で取得
    - レスポンスの各問題オブジェクトに `comments` 配列を含める
- コメント投稿時に自動的に `issues.updated_at` を更新
- トランザクション保証により整合性を確保
- Winston構造化ログによる監査証跡を記録
- マルチテナント分離により自組織の問題のみ取得可能

---

8. 実験データ記録API（Experiment Data Recording）

API設計書: api_experiment_data.md
ルートファイル: `backend/src/routes/experimentRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
8.1	POST	/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data	実験データ一括保存	✅ 実装済み	experimentController.saveBatchData
8.2	GET	/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data	テンプレート実績一覧取得	✅ 実装済み	experimentController.getTemplateData

ミドルウェアチェーン:

- すべて: `auth` → `tenantIsolation` → `permissionCheck`
- 権限: 8.1（experiment_worker のみ書込可）、8.2（experiment_worker 読取専用）

備考:

- 実験データのテンプレート単位保存
- DataGrid（2次元配列）、ネストプロパティに対応
- **API 8.1: 実験データ一括保存**

  - `project_status = 2`（タスク実行中）のプロジェクトのみ保存可能（それ以外は 403）
  - `records` 配列で複数プレースを一括保存（All or Nothing トランザクション保証）
  - `use_deemed_progress = true` の組織では保存時に `transition_data` のステータスを自動更新（みなし進捗）
  - 詳細仕様: api_experiment_data.md API #1 を参照

- **API 8.2: テンプレート実績一覧取得**
  - DataRecordDialog でダイアログを開いた際に既存入力済みデータを復元するために使用
  - クエリパラメータ `transitionId` を必須とする
  - S3/Swift の MaiML ファイルをパースしてペトリネット構造を解析
  - 指定トランジションに隣接するプレース（入力・出力の両方）のレコードを返却
  - 隣接プレースの実績データが 0 件の場合も 200 OK（MAIMLテンプレート定義から初期値フォールバック生成）
  - 詳細仕様: api_experiment_data.md API #2 を参照

- **トランジションステータス更新APIは進捗確認API（6.6）に移動**
- マルチテナント分離により自組織の実験データのみ操作可能

---

9. 部署管理API（Department Management）

API設計書: api_department_management.md **ルートファイル**: `backend/src/routes/departmentRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
9.1	GET	/api/v1/systemManagement/department	部署一覧取得	✅ 実装済み	departmentController.getDepartments
9.2	GET	/api/v1/systemManagement/department/download-csv	部署CSVダウンロード	✅ 実装済み	departmentController.downloadDepartmentCsv
9.3	POST	/api/v1/systemManagement/department/upload-csv	部署CSVアップロード	✅ 実装済み	departmentController.uploadDepartmentCsv
9.4	GET	/api/v1/systemManagement/department/:departmentCd	部署詳細取得	✅ 実装済み	departmentController.getDepartmentById

9.5	POST	/api/v1/systemManagement/department	部署新規作成	✅ 実装済み	departmentController.createDepartment
9.6	PUT	/api/v1/systemManagement/department/:departmentCd	部署更新	✅ 実装済み	departmentController.updateDepartment
9.7	PATCH	/api/v1/systemManagement/department/:departmentCd/status	部署アクティブ状態更新	✅ 実装済み	departmentController.updateDepartmentStatus

ミドルウェアチェーン:

- すべて: `authMiddleware`（JWTからorganizationIdを取得）
- 権限: system_admin

備考:

- ✅ 全エンドポイント実装完了
- マルチテナント分離（authMiddlewareがorganization_idを設定）
- 検索・フィルタリング機能（search, is_active）
- ソート機能（sort_by, sort_order）
- CSVアップロード時のバリデーション
- ファイルサイズ制限: 10MB

---

10. ロール管理API（Role Management）

API設計書: api_role_management.md **ルートファイル**: `backend/src/routes/roleRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
10.1	GET	/api/v1/master/role	ロール一覧取得	✅ 実装済み	roleController.getRoles
10.2	GET	/api/v1/master/role/download-csv	ロールCSVダウンロード	✅ 実装済み	roleController.downloadRoleCsv
10.3	GET	/api/v1/master/role/:roleId	ロール詳細取得	✅ 実装済み	roleController.getRoleById
10.4	POST	/api/v1/master/role	ロール新規追加	✅ 実装済み	roleController.createRole
10.5	POST	/api/v1/master/role/upload-csv	ロールCSVアップロード	✅ 実装済み	roleController.uploadRoleCsv

10.6	PUT	/api/v1/master/role/:roleId	ロール更新	✅ 実装済み	roleController.updateRole
10.7	PUT	/api/v1/master/role/:roleId/permissions	ロール権限更新	✅ 実装済み	roleController.updateRolePermissions
10.8	PATCH	/api/v1/master/role/:roleId/status	ロールアクティブ状態更新	✅ 実装済み	roleController.updateRoleStatus

ミドルウェアチェーン:

- すべて: `authMiddleware`
- 権限: system_admin, data_admin

備考:

- ✅ 全エンドポイント実装完了
- ロール権限管理機能（機能名、書き込み権限の設定）
- 初期画面機能名の設定
- マルチテナント分離
- CSVアップロード時のバリデーション

---

11. マスタ管理API（Master Data Management）

API設計書: api_master_management.md **ルートファイル**: `backend/src/routes/masterRoutes.ts`

11.1 Creatorマスタ（7エンドポイント）

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
11.1.1	GET	/api/v1/master/creator	Creator一覧取得	✅ 実装済み	masterController.getCreators
11.1.2	GET	/api/v1/master/creator/download-csv	CreatorCSVダウンロード	✅ 実装済み	masterController.downloadCreatorsCsv
11.1.3	POST	/api/v1/master/creator/upload-csv	CreatorCSVアップロード	✅ 実装済み	masterController.uploadCreatorsCsv
11.1.4	GET	/api/v1/master/creator/:creatorId	Creator詳細取得	✅ 実装済み	masterController.getCreatorById

11.1.5	POST	/api/v1/master/creator	Creator新規作成	✅ 実装済み	masterController.createCreator
11.1.6	PUT	/api/v1/master/creator/:creatorId	Creator更新	✅ 実装済み	masterController.updateCreator
11.1.7	PATCH	/api/v1/master/creator/:creatorId/status	Creatorアクティブ状態更新	✅ 実装済み	masterController.updateCreatorStatus

11.2 Vendorマスタ（7エンドポイント）

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
11.2.1	GET	/api/v1/master/vendor	Vendor一覧取得	✅ 実装済み	masterController.getVendors
11.2.2	GET	/api/v1/master/vendor/download-csv	VendorCSVダウンロード	✅ 実装済み	masterController.downloadVendorsCsv
11.2.3	POST	/api/v1/master/vendor/upload-csv	VendorCSVアップロード	✅ 実装済み	masterController.uploadVendorsCsv
11.2.4	GET	/api/v1/master/vendor/:vendorId	Vendor詳細取得	✅ 実装済み	masterController.getVendorById
11.2.5	POST	/api/v1/master/vendor	Vendor新規作成	✅ 実装済み	masterController.createVendor
11.2.6	PUT	/api/v1/master/vendor/:vendorId	Vendor更新	✅ 実装済み	masterController.updateVendor
11.2.7	PATCH	/api/v1/master/vendor/:vendorId/status	Vendorアクティブ状態更新	✅ 実装済み	masterController.updateVendorStatus

11.3 Instrumentマスタ（7エンドポイント）

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
11.3.1	GET	/api/v1/master/instrument	Instrument一覧取得	✅ 実装済み	masterController.getInstruments
11.3.2	GET	/api/v1/master/instrument/download-csv	InstrumentCSVダウンロード	✅ 実装済み	masterController.downloadInstrumentsCsv
11.3.3	POST	/api/v1/master/instrument/upload-csv	InstrumentCSVアップロード	✅ 実装済み	masterController.uploadInstrumentsCsv

11.3.4	GET	/api/v1/master/instrument/:instrumentId	Instrument詳細取得	✅ 実装済み	masterController.getInstrumentById
11.3.5	POST	/api/v1/master/instrument	Instrument新規作成	✅ 実装済み	masterController.createInstrument
11.3.6	PUT	/api/v1/master/instrument/:instrumentId	Instrument更新	✅ 実装済み	masterController.updateInstrument
11.3.7	PATCH	/api/v1/master/instrument/:instrumentId/status	Instrumentアクティブ状態更新	✅ 実装済み	masterController.updateInstrumentStatus

ミドルウェアチェーン:

- すべて: `authMiddleware`（JWTからorganizationIdを取得）
- 権限: system_admin, data_admin

備考:

- ✅ 全21エンドポイント実装完了
- マルチテナント分離（authMiddlewareがorganization_idを設定）
- Creatorマスタ: 表示スコープ設定（all: 全組織、own: 自組織のみ）
- 検索・フィルタリング機能（search, is_active, display_scope）
- CSVアップロード時のバリデーション
- ファイルサイズ制限: 10MB

---

12. 会社管理API（Corporation Management）

API設計書: api_corporation_management.md
ルートファイル: `backend/src/routes/corporationRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
12.1	GET	/api/v1/provider/corporation	会社一覧取得	✅ 実装済み	corporationController.getCorporation
12.2	GET	/api/v1/provider/corporation/download-csv	会社CSVダウンロード	✅ 実装済み	corporationController.downloadCorporationCsv
12.3	POST	/api/v1/provider/corporation/upload-csv	会社CSVアップロード	✅ 実装済み	corporationController.uploadCorporationCsv

12.4	GET	/api/v1/provider/corporation/:corporationId	会社詳細取得	✅ 実装済み	corporationController.getCorporationById
12.5	POST	/api/v1/provider/corporation	会社新規作成	✅ 実装済み	corporationController.createCorporation
12.6	PUT	/api/v1/provider/corporation/:corporationId	会社更新	✅ 実装済み	corporationController.updateCorporation
12.7	PATCH	/api/v1/provider/corporation/:corporationId/status	会社アクティブ状態更新	✅ 実装済み	corporationController.updateCorporationStatus

ミドルウェアチェーン:

- すべて: `authMiddleware`
- 権限: provider（システムプロバイダー）

備考:

- ✅ 全エンドポイント実装完了
- プロバイダー専用のマスタ管理API
- 検索・フィルタリング機能（search, is_active）
- ソート機能（sort_by, sort_order）
- CSVアップロード時のバリデーション
- ファイルサイズ制限: 10MB

---

13. ファイルアクセスAPI（File Access）

API設計書: api_file_access.md
ルートファイル: `backend/src/routes/fileAccessRoutes.ts`
備考: 実装済み7/7エンドポイント（100%）、全画面共通で使用

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
14.1	POST	/api/v1/files/upload	ファイルアップロード	✅ 実装済み	fileAccessController.uploadFiles
14.2	POST	/api/v1/files/download-folder-as-zip	フォルダZIPダウンロード	✅ 実装済み	fileAccessController.downloadFolderAsZip
14.3	POST	/api/v1/files/download-url	署名付きURL取得	✅ 実装済み	fileAccessController.getFileDownloadUrl

14.4	GET	/api/v1/projects/:projectId/files/:fileName/download	単一ファイルダウンロード（レガシー）	✅ 実装済み	fileAccessController.downloadFile
14.5	GET	/api/v1/files/batches	バッチ履歴取得	✅ 実装済み	fileAccessController.getBatchHistory
14.6	GET	/api/v1/files/batches/:batchId/progress	バッチ進捗取得	✅ 実装済み	fileAccessController.getBatchProgress

ミドルウェアチェーン:

- すべて: `authMiddleware`（JWTからorganizationIdを取得）
- upload.array('files'): ファイルアップロード用（14.1のみ）
- 権限: すべてのロール（マルチテナント分離で保護）

備考:

- ✅ 全エンドポイント実装完了
- Phase1-2: ファイルサイズ上限500MBに引き上げ（2026-02-18）
- Phase1-3: 空フォルダプレースホルダー機能追加（2026-02-18）
- Phase2-1: バッチ管理機能追加（2026-02-18）
- マルチテナント分離（organization_prefix必須）
- 汎用性: プロジェクト、フォーマット、その他あらゆるファイルで利用可能
- バッチ処理: 最大100ファイル/バッチ、500MB/ファイル
- 署名付きURL: 有効期限1時間（最大7日間）

---

14. 組織管理API（Organization Management）

API設計書: api_organization_management.md **ルートファイル**: `backend/src/routes/organizationRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
14.1	GET	/api/v1/provider/organization	組織一覧取得	✅ 実装済み	organizationController.getOrganizations
14.2	GET	/api/v1/provider/organization/download-csv	組織CSVダウンロード	✅ 実装済み	organizationController.downloadOrganizationCsv
14.3	POST	/api/v1/provider/organization/upload-csv	組織CSVアップロード	✅ 実装済み	organizationController.uploadOrganizationCsv

14.4	GET	/api/v1/provider/organization/:organizationId	組織詳細取得	✅ 実装済み	organizationController.getOrganizationById
14.5	GET	/api/v1/provider/organization/:organizationId/system-admins	システム管理者一覧取得	✅ 実装済み	organizationController.getSystemAdminsByOrganizationId
14.6	POST	/api/v1/provider/organization	組織新規作成	✅ 実装済み	organizationController.createOrganization
14.7	PUT	/api/v1/provider/organization/:organizationId	組織更新	✅ 実装済み	organizationController.updateOrganization
14.8	PATCH	/api/v1/provider/organization/:organizationId/status	組織アクティブ状態更新	✅ 実装済み	organizationController.updateOrganizationStatus
14.9	GET	/api/v1/provider/organization/:organizationId/certificate	クライアント証明書ダウンロード	✅ 実装済み	organizationController.downloadCertificate
14.10	GET	/api/v1/provider/organization/download-information-csv	情報CSVダウンロード	✅ 実装済み	organizationController.downloadInformationCsv

ミドルウェアチェーン:

- すべて: `authMiddleware`
- 権限: provider（システムプロバイダー）

備考:

- ✅ 全エンドポイント実装完了
- プロバイダー専用の組織管理API
- 組織新規作成時に関連4テーブルへ自動挿入（organization, organization_user, role, role_permission）
- 検索・フィルタリング機能（search, is_active, organization_cd, organization_display_name, corporation_name）
- ソート機能（sort_by, sort_order）
- CSVアップロード：組織追加ダイアログと同じ処理を各レコードごとに実行
- ファイルサイズ制限: 10MB

---

15. システム概要API（System Overview）

API設計書: api_system_overview.md
ルートファイル: `backend/src/routes/systemOverviewRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
15.1	GET	/api/v1/systemManagement/system-overview	システム概要統計取得	✅ 実装済み	systemOverviewController.getOverview

ミドルウェアチェーン:

- `authMiddleware` → `tenantIsolation`
- 権限: system_admin

備考:

- ✅ 実装済み
- ユーザー統計・セッション数・実験システム状況・ストレージ・ログイン履歴を一括取得
- 5つのデータソース（DB×3、Redis、Swift/S3）を `Promise.all` で並列取得
- セッション・ストレージ取得失敗時は `0` を返却し画面表示を継続
- `experimentSystem.planning`: `project_status` IN `'0'`, `'1'`（プロジェクト計画中/タスク実行待ち）
- `experimentSystem.inProgress`: `project_status` IN `'2'`, `'3'`（タスク実行中/タスク結果レビュー待ち）
- `experimentSystem.completed`: `project_status` IN `'4'`, `'5'`（プロジェクト完了/タスク異常終了）
- ストレージ統計: Swift/S3から直接取得（`storageService.listFiles()`）。フォルダオブジェクト・`.keep` プレースホルダーを除外して集計。`usedCapacityGB` は小数点3桁（GB単位）

---

16. セマンティック検索API（Semantic Search）

API設計書: api_semantic_search.md
ルートファイル: `backend/src/routes/semanticSearchRoutes.ts`
備考: 実装済み9/9エンドポイント（100%）、4軸ベクトル検索

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
16.1	POST	/api/v1/semantic-search/search	セマンティック検索	✅ 実装済み	semanticSearchController.search

16.2	POST	/api/v1/semantic-search/decompose	クエリ分解（AI4軸分解）	✅ 実装済み	semanticSearchController.decompose
16.3	POST	/api/v1/semantic-search/index	ファイルインデックス登録	✅ 実装済み	semanticSearchController.indexFile
16.4	GET	/api/v1/semantic-search/stats	インデックス統計取得	✅ 実装済み	semanticSearchController.getStats
16.5	GET	/api/v1/semantic-search/health	ヘルスチェック	✅ 実装済み	semanticSearchController.health
16.6	POST	/api/v1/semantic-search/batch	バッチ処理開始	✅ 実装済み	semanticSearchController.startBatch
16.7	GET	/api/v1/semantic-search/batch	バッチジョブ一覧取得	✅ 実装済み	semanticSearchController.listBatchJobs
16.8	GET	/api/v1/semantic-search/batch/stats	バッチ統計取得	✅ 実装済み	semanticSearchController.getBatchStats
16.9	DELETE	/api/v1/semantic-search/batch/:jobId	バッチジョブキャンセル	✅ 実装済み	semanticSearchController.cancelBatchJob

ミドルウェアチェーン:

- すべて: `authMiddleware`（JWTからorganizationIdを取得）
- 権限: すべてのロール（マルチテナント分離で保護）

備考:

- 4軸ベクトル検索（入力条件・手順・出力・構造）によりMAIMLトランジションを類似度評価
- AI（GPT）による自然言語クエリの4軸自動分解（GPT未接続時はフォールバック動作）
- 検索重み調整: 4軸ごとに0.0〜1.0の重みをカスタマイズ可能
- バッチ処理: 組織内全MAIMLファイルの一括Embedding生成（非同期実行）
- 使用コンポーネント: `TransitionSetSearchModal.tsx`、`TransitionSetBatchProcessingPanel.tsx`
- 呼び出し元サービス: `frontend/src/services/semanticSearchService.ts`
- 詳細仕様: api_semantic_search.md を参照

---

17. ユーザー管理API（User Management）

API設計書: api_user_management.md
ルートファイル: `backend/src/routes/userRoutes.ts`

No	HTTPメソッド	エンドポイント	説明	実装状態	コントローラー
15.1	GET	/api/v1/systemManagement/user	ユーザー一覧取得	✅ 実装済み	userController.getUsers
15.2	GET	/api/v1/systemManagement/user/:userId	ユーザー詳細取得	✅ 実装済み	userController.getUserById
15.3	POST	/api/v1/systemManagement/user	ユーザー新規作成	✅ 実装済み	userController.createUser
15.4	PUT	/api/v1/systemManagement/user/:userId	ユーザー更新	✅ 実装済み	userController.updateUser
15.5	PATCH	/api/v1/systemManagement/user/:userId/status	アクティブ状態更新	✅ 実装済み	userController.updateUserStatus
15.6	GET	/api/v1/systemManagement/user/download-csv	CSVダウンロード	✅ 実装済み	userController.downloadUsersCsv
15.7	POST	/api/v1/systemManagement/user/upload-csv	CSVアップロード	✅ 実装済み	userController.uploadUsersCsv

ミドルウェアチェーン:

- すべて: `authMiddleware`（JWTからorganizationIdを取得）
- 権限: system_admin

備考:

- ✅ 全エンドポイント実装完了（100%）
- パスワード管理: bcryptハッシュ化、8文字以上必須
- 最終ログイン表示: login_historyテーブルから最新ログイン日時を取得
- CSVダウンロード: パスワード欄は空白で出力
- CSVアップロード: パスワード必須（user_authenticationテーブルに登録）
- マルチテナント分離（authMiddlewareがorganization_idを設定）
- 検索・フィルタリング機能（search, is_active, department_cd, role_cd）
- ソート機能（sort_by, sort_order）
- ファイルサイズ制限: 10MB

🔍 API検索ガイド

API名で検索する

例: "プロジェクト一覧を取得したい"

1. このドキュメントで「プロジェクト一覧取得」を検索
2. API番号: 2.1
3. ルートファイル: `backend/src/routes/projectRoutes.ts`
4. コントローラー: `ProjectController.list`
5. API設計書: api_project_management.md のセクション2.1

エンドポイントで検索する

例: "POST /api/v1/auth/login の実装を確認したい"

1. このドキュメントで `/api/v1/auth/login` を検索
2. API番号: 1.1
3. ルートファイル: `backend/src/routes/authRoutes.ts`
4. コントローラー: `AuthController.login`
5. API設計書: api_auth.md のセクション3.1

機能カテゴリで探す

認証が必要な場合:

- セクション1: 認証API

プロジェクト管理が必要な場合:

- セクション2: プロジェクト管理API

ファイル操作が必要な場合:

- セクション3: 共通API（ファイルアップロード/ダウンロード）
- セクション4: フォーマット管理API

- セクション5: Maiml編集API

進捗確認が必要な場合:

- セクション6: 進捗確認API

📊 実装状況サマリー

全体統計

カテゴリ	総数	実装済み	スタブ	未実装	実装率
認証API	6	6	0	0	100%
プロジェクト管理API	22	19	0	3	86%
共通API	2	0	0	2	0%
フォーマット管理API	6	5	0	1	83%
Maiml編集API	18	17	0	1	94%
進捗確認API	7	7	0	0	100%
問題管理API	5	5	0	0	100%
実験データ記録API	2	2	0	0	100%
部署管理API	7	7	0	0	100%
ロール管理API	8	8	0	0	100%
マスタ管理API	21	21	0	0	100%
会社管理API	7	7	0	0	100%
ファイルアクセスAPI	6	6	0	0	100%
組織管理API	10	10	0	0	100%
システム概要API	1	1	0	0	100%
ユーザー管理API	7	7	0	0	100%
MaiML OpenData Pipeline API	3	3	0	0	100%
合計	**134**	**127**	**0**	**14**	**89%**

優先度別実装状況

Phase 1（基本機能）- 完了:

- ✅ 認証API（6エンドポイント）
- ✅ プロジェクト一覧取得
- ✅ プロジェクト詳細取得
- ✅ プロジェクト作成
- ✅ 指示送信

Phase 2（検索・管理機能）- 完了:

- ✅ プロジェクト詳細検索

Phase 3（ファイル管理）- 未実装:

- ⏳ ファイルアップロード/ダウンロード（共通API）
- ⏳ フォーマット管理API（5エンドポイント）
- ⏳ Maiml編集API（8エンドポイント）
- ⏳ チェックイン/チェックアウト

Phase 4（進捗・公開機能）- 未実装:

- ⏳ 進捗確認API（2エンドポイント）
- ⏳ プロジェクト公開

🛠️ 実装ファイルパス一覧

ルートファイル

```
backend/src/routes/
├── authRoutes.ts                    # 認証API（6エンドポイント）✅ 実装済み
├── projectRoutes.ts                 # プロジェクト管理API（9エンドポイント + 指示確認完了）✅
一部実装済み
├── fileRoutes.ts                    # 共通API（2エンドポイント）⏳ 未実装
├── formatRoutes.ts                  # フォーマット管理API（5エンドポイント）⏳ 未実装
├── maimlRoutes.ts                   # Maiml編集API（11エンドポイント）✅ 実装済み
```

```
├── progressRoutes.ts                  # 進捗確認API（4エンドポイント）⏳ 未実装
├── issueRoutes.ts                     # 問題管理API（5エンドポイント）✅ 実装済み
├── experimentDataRoutes.ts            # 実験データ記録API（2エンドポイント）⏳ 未実装
└── taskRoutes.ts                      # タスク管理API（1エンドポイント）✅ 実装済み
```

コントローラー

```
backend/src/controllers/
├── authController.ts                  # 認証コントローラー
├── projectController.ts               # プロジェクト管理コントローラー
├── maimlController.ts                 # Maiml編集コントローラー
├── issueController.ts                 # 問題管理コントローラー
├── taskController.ts                  # タスク管理コントローラー
└── instructionConfirmationController.ts   # 指示確認コントローラー
```

サービス層

```
backend/src/services/
├── authService.ts                     # 認証サービス
├── projectService.ts                  # プロジェクト管理サービス
├── maimlService.ts                    # Maiml編集サービス
├── issueService.ts                    # 問題管理サービス
├── taskService.ts                     # タスク管理サービス
├── instructionConfirmationService.ts      # 指示確認サービス
└── storageService.ts                  # ストレージサービス（S3/Swift）
```

リポジトリ層

```
backend/src/repositories/
├── authRepository.ts                  # 認証リポジトリ
├── projectRepository.ts               # プロジェクト管理リポジトリ
├── issueRepository.ts                 # 問題管理リポジトリ
├── taskRepository.ts                  # タスク管理リポジトリ
└── instructionConfirmationRepository.ts   # 指示確認リポジトリ
```

⚠️ 重要な設計原則

マルチテナント分離（必須）

すべてのAPIで以下を徹底：

```
// ✅ 良い例
const projects = await db.query(
  'SELECT * FROM projects WHERE organization_id = $1',
  [organizationId]
);

// ❌ 悪い例
const projects = await db.query('SELECT * FROM projects');
```

ミドルウェアチェーン（必須）

すべての認証付きAPIで以下の順序を適用：

```
router.get('/projects',
  auth,                          // 1. JWT認証
  tenantIsolation,               // 2. テナント分離
  permissionCheck(['manager']), // 3. 権限チェック
  projectController.list         // 4. ルートハンドラ
);
```

S3操作（必須）

organization_prefix を必ず含める：

```
// ✅ 良い例
const path = `org_${organizationPrefix}/projects/${projectName}/file.maiml`;

// ❌ 悪い例
const path = `projects/${projectName}/file.maiml`;
```

RESTful設計原則

- リソース指向のURL設計

- 適切なHTTPメソッド使用（GET/POST/PUT/PATCH/DELETE）
- 動詞はサブリソースとして配置（例: `/maiml/parse`, `/maiml/validate`）
- kebab-case統一（例: `/api/v1/maiml-files`）

📝 更新ガイドライン

新規API追加時

1. **API設計書を作成または更新**
   - `docs/design/api/api_*.md` に仕様を記載
2. **このドキュメントを更新**
   - 該当カテゴリのテーブルに追加
   - 実装状況サマリーを更新
3. **実装ファイルを作成**
   - ルート、コントローラー、サービスを実装
4. **テストを追加**
   - `backend/tests/` にテストケースを追加

API仕様変更時

1. **API設計書を更新**
   - 変更内容と変更履歴を記録
2. **このドキュメントを更新**
   - 説明欄を更新（必要に応じて）
3. **実装を修正**

- ルート、コントローラー、サービスを修正

4. **テストを更新**
   - 変更に合わせてテストケースを修正

🔗 関連ドキュメント

規約

- API設計ポリシー - API設計の統一ルール
- バックエンド規約 - 実装規約
- セキュリティ規約 - セキュリティ要件

参照ドキュメント

- API構成定義 - API構成の参照ルール
- データベース構造 - DB設計
- S3構造 - S3フォルダ構成

API設計書

- 認証API - 認証・セッション管理
- プロジェクト管理API - プロジェクト操作
- 共通API - ファイルアップロード/ダウンロード
- フォーマット管理API - フォーマット管理
- Maiml編集API - MAIML編集・パース
- 進捗確認API - タスク進捗確認

📋 変更履歴

バージョン	日付	変更内容	担当者
2.18.0	2026-03-12	進捗確認API・実験データ記録API・プロジェクト管理APIの実装状況を更新	Cline
		- 進捗確認API: 6.3〜6.6 を実装済みに更新（6.3: getProgressData, 6.4: getWorkerTasks, 6.5: updateTaskStatus, 6.6: updateTransitionStatus）	
		- 実験データ記録API: 8.1/8.2 を実装済みに更新、ルートファイルを experimentRoutes.ts に修正	
		- プロジェクト管理API: 2.13/2.15/2.16/2.17 を実装済みに更新、2.17 エンドポイントを :projectName に修正	
		- 実装状況サマリー: 進捗確認API 43%→100%、実験データ記録API 0%→100%、プロジェクト管理API 81%→90%	
		- 全体: 113/127 (89%) → 124/131 (95%)	
2.17.0	2026-03-11	進捗確認API - 作業詳細一括取得API追加	Cline
		- API 6.7: GET /api/v1/progress/projects/:projectName/work-details 新規追加（実装済み）	
		- WorkProgressDetailDialog 用の projectInfo / fileInfo / workDetails を一括返却	
		- 権限: experiment_admin（040000 読取専用以上）	
		- 進捗確認API: 6→7エンドポイント（実装済み2→3）	
		- 全体: 126→127エンドポイント、実装済み112→113	
		- 詳細仕様: api_progress_monitoring.md API #7を参照	
2.16.0	2026-03-06	Maiml編集API - 名前空間テンプレートフォルダ＆ファイル一覧取得API追加	Cline
		- API 5.18: GET /api/v1/maiml/formats 新規追加（未実装）	
		- 名前空間テンプレート取り込み機能（editorv2 TemplateEditorDialog）向け	
		- 全部門横断でmaterial/condition/result配下の.xmlファイル一覧を返す	
		- Maiml編集API: 17→18エンドポイント（実装済み17、未実装1）	
		- 全体: 124→125エンドポイント	
		- 詳細仕様: api_maiml_editor.md API #18を参照	
2.15.0	2026-02-27	システム概要API - ストレージ統計仕様変更	Cline
		- ストレージ統計取得元をSwift/S3直接取得（ `storageService.listFiles()` ）に変更	

		- `usedCapacityGB` の小数点桁数を2桁→3桁に変更	
2.14.0	2026-02-26	システム概要API・情報CSVダウンロードAPI追加	Cline
		- セクション15: システム概要API新規追加（1エンドポイント、実装済み100%）	
		- API 15.1: GET /api/v1/systemManagement/system-overview	
		- 組織管理API: 9→10エンドポイント（API 14.10: 情報CSVダウンロード追加）	
		- API設計書: api_system_overview.md 新規作成	
		- API設計書: api_information_download.md 新規作成	
		- 実装済みAPI総数: 108→110エンドポイント（+2）	
		- 全体: 122→124エンドポイント	
		- 全体実装率: 88%→89%に向上	
2.13.0	2026-02-25	実験データ記録API - テンプレート実績一覧取得API追加	Cline
		- API 8.2: GET .../tasks/:taskNo/data 追加	
		- DataRecordDialog でのモックデータ（templatesList）置き換え用	
		- transitionId（クエリ）で MaiML ペトリネット解析し隣接プレースを特定	
		- 実験データ記録API: 1→2エンドポイント（+1: テンプレート実績一覧取得）	
		- 実装状況サマリー: 実験データ記録API 総数 1→2	
		- 全体: 115→116エンドポイント	
		- 詳細仕様: api_experiment_data.md セクション2 を参照	
2.12.0	2026-02-24	フォーマット管理API - 新規作成機能実装完了	
		- PKCS#12形式の証明書ダウンロード	
		- 組織管理API: 8→9エンドポイント	
		- 実装済みAPI総数: 81→82エンドポイント（+1）	
		- 全体: 103→104エンドポイント	
		- 実装率: 79%維持	
2.10.1	2026-02-19 17:20	14番項番のAPI競合を整理（115エンドポイント）	Cline
		- 組織管理APIを14番から15番に移動（API番号14.1-14.8→15.1-15.8）	
		- カテゴリ数: 14→15カテゴリ	

		- 合計エンドポイント数: 107→115（組織管理API 8エンドポイントを追加計上）	
		- 全体実装率: 85%→86%に微増	
2.10.0	2026-02-19 17:10	routeファイル実装状況を反映（107エンドポイント）トランジションステータス更新API移動（96エンドポイント）	Cline
		- プロジェクト管理API: 17→16エンドポイント、実装済み13個（81%）	
		- Maiml編集API: 10→17エンドポイント、実装済み17個（100%完了）	
		- Draft機能追加（4エンドポイント）、ファイル管理追加（3エンドポイント）	
		- ファイルアクセスAPI: 新規カテゴリ追加、6エンドポイント（実装済み100%）	
		- ファイルアップロード、フォルダZIPダウンロード、署名付きURL取得	
		- バッチ管理機能（履歴・進捗取得）	
		- カテゴリ数: 13→14カテゴリ	
		- 実装済みAPI総数: 73→91エンドポイント（+18エンドポイント）	
		- 全体実装率: 77%→85%に向上	
		- 進捗確認API: 5→6エンドポイント（+1: トランジションステータス更新）	
		- API 6.6: PATCH /api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status	
		- 実験データ記録API: 2→1エンドポイント（-1: トランジションステータス更新を進捗確認APIに移動）	
		- taskNoをリクエストボディパラメータ化	
		- 権限: experiment_worker（書込可）	
		- 総エンドポイント数: 95→96（+1）	
		- 未実装API数: 22→23（+1）	
		- 全体実装率: 76%→75%（分母増加により一時的に低下）	
2.9.0	2026-02-18	トランジションステータス更新API移動（96エンドポイント）	Cline
2.9.1	2026-02-19	フォーマット管理API一部実装（103エンドポイント）	Cline
		- フォーマット管理API: 0→2エンドポイント実装済み（40%）	
		- API 4.2: フォーマットファイル一覧取得を実装	
		- サブフォルダ対応（pathクエリパラメータ）	
		- ファイルとフォルダを区別して返却	

		- フォルダには配下のファイル数（fileCount）を含む	
		- 実装済みAPI総数: 81→83エンドポイント（+2エンドポイント）	
		- 全体実装率: 79%→81%に向上	
2.8.4	2026-02-18	タスク一覧取得（実験作業者用）API仕様変更（95エンドポイント）	Cline
		- 進捗確認API: 6→5エンドポイント（置き換え）	
		- API 6.4: GET /api/v1/progress/projects/:projectName/worker-tasks（エンドポイント変更）	
		- タスク一覧取得（プロジェクト単位）からタスク一覧取得（実験作業者用）に変更	
		- プロジェクト名のみで特定（methodIdパラメータ削除）	
		- 総エンドポイント数: 96→95（-1）	
		- 実装率: 75%→76%に向上	
2.8.3	2026-02-18	タスク一覧取得（実験作業者用）API追加（96エンドポイント）	Cline
		- 進捗確認API: 5→6エンドポイント（+1: タスク一覧取得（実験作業者用））	
		- API 6.6: GET /api/v1/progress/projects/:projectName/methods/:methodId/worker-tasks	
		- 実験作業者が自分に割り当てられたタスクの一覧を取得	
		- 権限: experiment_worker（読取専用）	
		- 総エンドポイント数: 95→96（+1）	
		- 未実装API数: 22→23（+1）	
		- 全体実装率: 76%→75%（分母増加により一時的に低下）	
2.8.2	2026-02-18	タスク管理APIセクション削除（95エンドポイント）	Cline
		- タスク管理API（セクション9）を完全削除	
		- 後続セクションを繰り上げ（10→9、11→10、12→11、13→12）	
		- 総エンドポイント数: 96→95（-1: タスクステータス更新旧API削除）	
		- 実装済みAPI数: 73→72	
		- 進捗確認API 6.5が新しいタスクステータス更新APIとして機能	
2.8.1	2026-02-18	タスクステータス更新APIを進捗確認APIに追加（96エンドポイント）	Cline
		- 進捗確認API: 4→5エンドポイント（+1: タスクステータス更新）	

		- API 6.5: PATCH /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/status	
		- タスク管理API 9.1は旧エンドポイントとして非推奨化	
		- 全体実装率: 77%→76%（分母増加により一時的に低下）	
2.8.0	2026-02-13	部署・ロール・マスタ・会社管理API追加（95エンドポイント）	Cline
		- 部署管理API: 新規カテゴリ追加、7エンドポイント（実装済み100%）	
		- ロール管理API: 新規カテゴリ追加、8エンドポイント（実装済み100%）	
		- マスタ管理API: 新規カテゴリ追加、21エンドポイント（実装済み100%）	
		- Creator: 7エンドポイント、Vendor: 7エンドポイント、Instrument: 7エンドポイント	
		- 会社管理API: 新規カテゴリ追加、7エンドポイント（実装済み100%）	
		- カテゴリ数: 9→13カテゴリ	
		- 実装済みAPI総数: 30→73エンドポイント（+43エンドポイント）	
		- 全体実装率: 56%→77%に大幅向上	
2.7.0	2026-01-31	指示確認API実装完了	Development Team
2.6.0	2026-01-28 14:45	タスク管理API追加（52→53エンドポイント）	Cline
		- 新規カテゴリ追加: タスク管理API（1エンドポイント、実装済み100%）	
		- 進捗確認API: 5→4エンドポイント（6.5タスクステータス更新を削除）	
		- カテゴリ数: 8→9カテゴリ	
		- 実装済みAPI総数: 28→29エンドポイント	
		- 全体実装率: 54%→56%に向上	
		- API 9.1: タスクステータス更新（楽観的ロック、並行更新制御実装済み）	
2.5.2	2026-01-27 19:43	API設計書リンク更新（問題管理・実験データ記録）	Cline
		- 問題管理API: 設計書リンクをapi_issue_management.mdに更新	
		- 実験データ記録API: 設計書リンクをapi_experiment_data.mdに更新	
		- API一覧_実験作業者画面.mdからの移行完了	
2.5.1	2026-01-27 18:51	実験作業者用プロジェクト一覧取得API追加（52エンドポイント）	Cline

		- プロジェクト管理API: 15→16エンドポイント（+1: 実験作業者用プロジェクト一覧取得）	
		- 全体実装率: 55%→54%（分母増加により一時的に低下）	
2.5.0	2026-01-27 18:33	実験作業者API統合（51エンドポイント）	Cline
		- プロジェクト管理API: 12→15エンドポイント（+3: 指示管理2個、承認依頼1個）	
		- Maiml編集API: 10→11エンドポイント（+1: MAIMLファイル取得）	
		- 進捗確認API: 2→5エンドポイント（+3: 進捗データ、タスク一覧・更新）	
		- 問題管理API: 新規カテゴリ追加、5エンドポイント（実装済み100%）	
		- 実験データ記録API: 新規カテゴリ追加、2エンドポイント（未実装）	
		- カテゴリ数: 6→8カテゴリ	
		- 全体実装率: 62%→55%（分母増加により一時的に低下）	
		- 実装済みAPI総数: 23→28エンドポイント（+5: 問題管理API）	
2.4.0	2026-01-27 09:24	Maiml編集API実装完了（37エンドポイント）	Cline
		- Maiml編集API: 10エンドポイント、9実装済み、1スタブ（90%）	
		- 新規実装: parse, parsePetriNetStructure, save, load, list	
		- 新規実装: validate, session/start, session/end, health	
		- スタブ実装: correct（自動補正機能）	
		- Phase 4（2026-01-27）: サービス層実装完了	
		- Phase 6（2026-01-27）: API接続完全統合	
		- 全体実装率: 40%→62%に大幅向上	
2.3.0	2026-01-26 15:42	プロジェクト管理APIの実装状況を更新（35エンドポイント）	Cline
		- プロジェクト管理API: 12エンドポイント、8実装済み（67%）	
		- 新規実装: プロジェクト詳細取得、チェックアウト状態取得	
		- 新規実装: ファイル一覧取得、タスク一覧取得	
		- 全体実装率: 30%→40%に向上	
		- 未実装: 指示送信・一覧、プロジェクト詳細検索、公開	
2.2.0	2026-01-26	プロジェクトリロードAPI削除（33エンドポイント）	Cline

		- プロジェクト管理API: 11→10エンドポイントに削減	
		- `/api/v1/projects/:projectId/reload` エンドポイント削除	
		- 理由: 既存のプロジェクト一覧・ファイル一覧・タスク一覧APIで代用可能	
		- フロントエンドは既存APIを組み合わせたリロード処理に変更済み	
2.1.0	2026-01-23	プロジェクト管理APIを最新化（34エンドポイント）	Cline
		- プロジェクト管理API: 10→11エンドポイントに更新	
		- チェックアウト/チェックイン機能を追加	
		- ファイル一覧/タスク一覧/指示一覧取得を追加	
		- プロジェクト詳細取得を削除（設計書に未記載）	
2.0.0	2026-01-23	全API仕様書を反映して大幅更新（33エンドポイント登録）	Cline
		- 共通API追加（2エンドポイント）	
		- フォーマット管理API追加（5エンドポイント）	
		- Maiml編集API追加（8エンドポイント）	
		- 進捗確認API追加（2エンドポイント）	
		- 実装状況サマリーを更新	
		- RESTful設計原則を追記	
1.0.0	2026-01-20	初版作成（17エンドポイント登録）	Development Team

注意: このドキュメントは実装状況に合わせて定期的に更新してください。新しいAPIを追加した際は、必ずこのドキュメントも更新してください。